

Unit-II

1. What is TensorFlow? What are the Components of TensorFlow?

- ❖ TensorFlow is a popular framework of machine learning and deep learning. It is a free and open-source library which is released on 9 November 2015 and developed by Google Brain Team. It is entirely based on Python programming language and use for **numerical computation and data flow**, which makes machine learning faster and easier.
- ❖ TensorFlow is the better library for all because it is accessible to everyone. TensorFlow library integrates different API to create a scale deep learning architecture like CNN (Convolutional Neural Network) or RNN (Recurrent Neural Network).
- ❖ **TensorFlow is based on graph computation; it can allow the developer to create the construction of the neural network with Tensorboard. This tool helps debug our program. It runs on CPU (Central Processing Unit) and GPU (Graphical Processing Unit).**
- ❖ TensorFlow can train and run the deep neural networks for image recognition, handwritten digit classification, recurrent neural network, word embedding, natural language processing, video detection, and many more. TensorFlow is run on multiple CPUs or GPUs (Graphics processing unit) and also mobile operating systems.
- ❖ The word TensorFlow is made by two words, i.e., Tensor and Flow
 - ❖ **Tensor is a multidimensional array**
 - ❖ **Flow is used to define the flow of data in operation.**
- ❖ **TensorFlow is used to define the flow of data in operation on a multidimensional array or Tensor**

Components of TensorFlow

Tensor

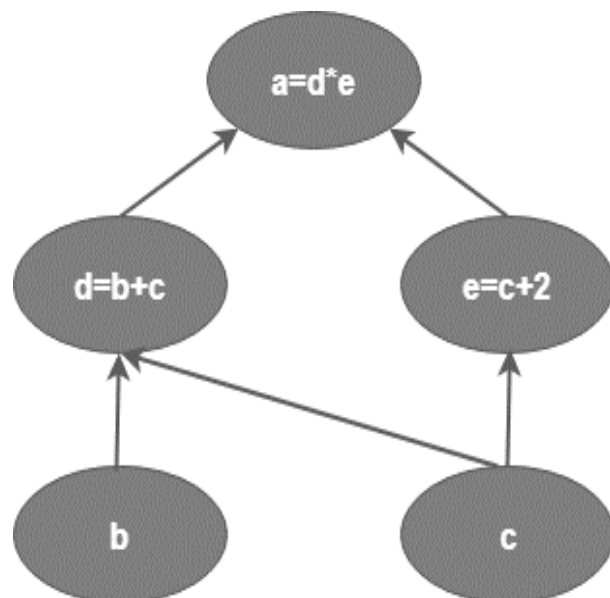
- ❖ The name TensorFlow is derived from its core framework, "Tensor." A tensor is a vector or a matrix of n-dimensional that represents all type of data. All values in a tensor hold similar data type with a known shape. The shape of the data is the dimension of the matrix or an array.
- ❖ A tensor can be generated from the input data or the result of a computation. In TensorFlow, all operations are conducted inside a graph. The group is a set of calculation that takes place successively. Each transaction is called an **op node** are connected.

Graphs

- ❖ TensorFlow makes use of a graph framework. The chart gathers and describes all the computations done during the training. **Consider the following expression $a = (b+c)*(c+2)$** , We can break the functions into components given below:

$$d=b+c, e=c+2, a=d*e$$

Now, we can represent these operations graphically below



Session

- ❖ A session can execute the operation from the graph. To feed the graph with the value of a tensor, we need to open a session. Inside a session, we must run an operator to create an output.

2. How to perform Linear Regression in TensorFlow, and Provide a step-by-step guide for the process?

- ❖ Linear regression is a fundamental machine learning algorithm used for modeling the relationship between a dependent variable (target) and one or more independent variables (features) by fitting a linear equation.
- ❖ TensorFlow is a popular deep learning framework that can be used to implement linear regression models as well. Here's a step-by-step guide on how to perform linear regression in TensorFlow:

Import Libraries:

- ❖ First, you need to import the necessary libraries, including TensorFlow and any other libraries you might need for data manipulation and visualization.

1. `import tensorflow as tf`
2. `import numpy as np`
3. `import matplotlib.pyplot as plt`

Prepare Data:

- ❖ You'll need to have your data in the form of NumPy arrays or TensorFlow tensors. Let's create some synthetic data for this example:

1. `# Generate some synthetic data`
2. `np.random.seed(0)`

3. `X = np.random.rand(100, 1)`

4. `y = 2 * X + 1 + 0.1 * np.random.randn(100, 1)`

Build the Model:

❖ In TensorFlow, you can create a linear regression model using the `tf.keras.Sequential` API. This is a simple single-layer model with one neuron.

```
1. model = tf.keras.Sequential([  
2.     tf.keras.layers.Input(shape=(1,)),  
3.     tf.keras.layers.Dense(1) # One neuron for linear regression  
4. ])
```

Compile the Model:

❖ Before training, you need to compile the model by specifying the loss function and optimizer. For linear regression, Mean Squared Error (MSE) is a common choice for the loss function.

```
1. model.compile(loss='mean_squared_error', optimizer='sgd')
```

Train the Model:

❖ Now, you can train the model using your data.

```
1. history = model.fit(X, y, epochs=100, verbose=1)
```

Evaluate the Model:

❖ After training, you can evaluate the model's performance and see how well it fits the data.

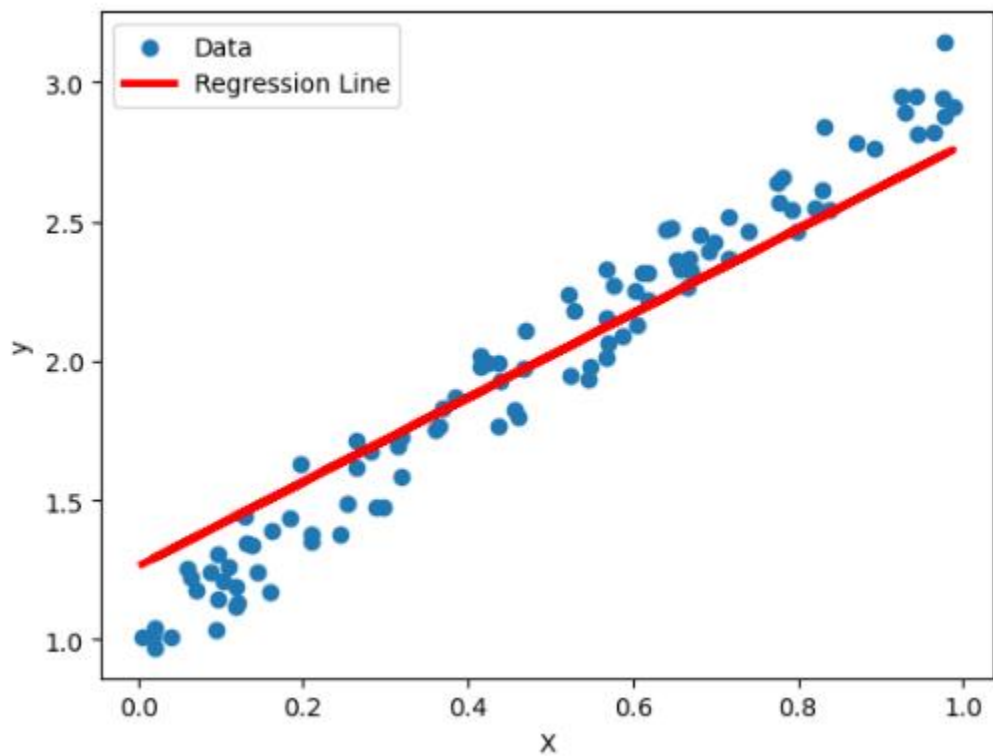
```
1. # Get the model's predictions  
2. predictions = model.predict(X)  
3. # Calculate the MSE on the training data  
4. mse = np.mean((predictions - y)**2)
```

5. `print(f"Mean Squared Error: {mse}")`

Visualize the Results:

❖ You can visualize the training process and the model's predictions.

1. `plt.scatter(X, y, label='Data')`
2. `plt.plot(X, predictions, color='red', linewidth=3, label='Regression Line')`
3. `plt.legend()`
4. `plt.xlabel('X')`
5. `plt.ylabel('y')`
6. `plt.show()`



3. Explain Perceptron with Activation Functions: ReLU

- ❖ A perceptron is one of the simplest artificial neural network architectures, and it serves as the fundamental building block of more complex neural networks.
- ❖ It was developed by Frank Rosenblatt in the late 1950s.
- ❖ **A perceptron is primarily used for binary classification tasks, where it takes a set of input values, applies weights to those inputs, sums them up, and then passes the result through an activation function to produce an output. This output is typically either 0 or 1, making it suitable for binary decisions.**

What is Perceptron and how it has been developed?

- ❖ The word Perceptron is first ever coined in **1943 by Warren McCulloch and Walter Pitts. Lately, Frank Rosenblatt**, an American psychologist, and computer scientist built the first Perceptron machine when doing research at **Cornell Aeronautical Laboratory** for performing image recognition.
- ❖ During this time the Perceptron machine become popular among the AI community and is considered the fundamental part of building intelligent systems.
- ❖ **The perceptron algorithm is based on the concept of a single neuron in the human brain, a single neuron in the human brain is doing a very simple thing, just receiving some inputs and if the inputs are high, it activates and send the signal to the next neuron.**
- ❖ The perceptron was designed to **mimic (copycat) this process**, with the input data serving as the input to the neuron and the weights representing the strength of the connections between the input neurons and the output neuron.

Perceptron with Activation Functions: ReLU

- ❖ A perceptron with the Rectified Linear Unit (ReLU) activation function is a type of artificial neural network unit that uses the ReLU function as its activation function. ReLU is one of the most commonly used activation functions in modern neural networks, including deep learning architectures.
- ❖ Here's an explanation of the perceptron with ReLU activation:
- ❖ **Input Layer:** Similar to a standard perceptron, a perceptron with ReLU activation takes one or more input values. These inputs are typically real-valued features or the outputs of other neurons in the network.
- ❖ **Weights and Bias:** Each input is associated with a weight, which represents the strength of the connection between the input and the neuron. Additionally, there is a bias term, which allows the model to shift the decision boundary.
- ❖ **Weighted Sum:** The perceptron calculates the weighted sum of the inputs by multiplying each input by its corresponding weight and then summing them up, just like in a standard perceptron:

$$z = \sum_{i=1}^n (x_i \cdot w_i) + b$$

Where:

- ❖ z is the weighted sum,
- ❖ x_i are the input values,
- ❖ w_i are the corresponding weights,
- ❖ n is the number of inputs, and
- ❖ b is the bias term.

- ❖ The ReLU function works by applying a simple mathematical operation to the input value:
- ❖ **If the input value is greater than or equal to zero: The output is equal to the input value.**
- ❖ **If the input value is negative: The output is zero**
- ❖ ReLU introduces non-linearity to a deep learning model and **solves the vanishing gradients issue**. It returns 0 if the input is negative, but for any positive input, it returns that value back. The function can be written as
- ❖ $f(x)=\max(0,x)$.

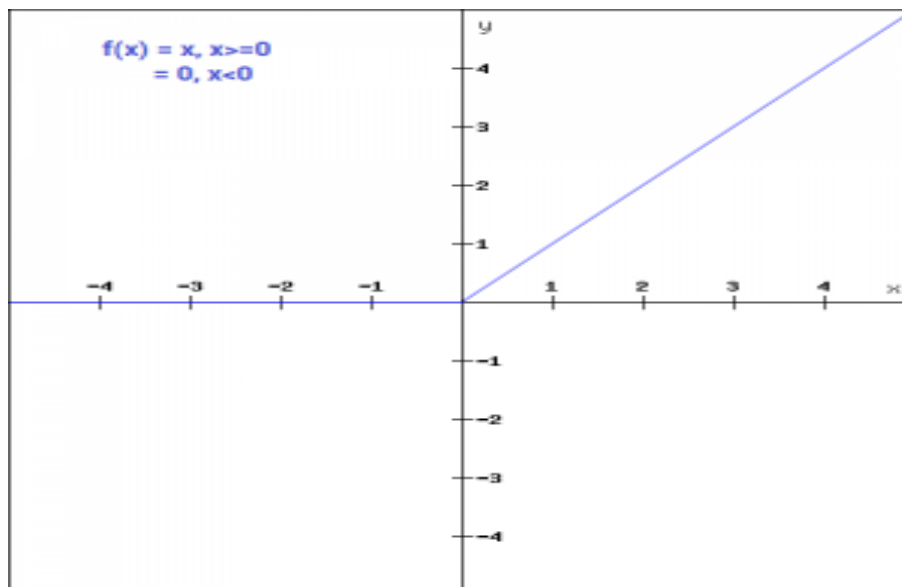
ReLU has several advantages, including:

- ❖ **Training complex models:** ReLU can train complex models and neural networks with constant values.
- ❖ **Efficiency:** ReLU runs faster than sigmoid functions and has a lower run time.
- ❖ **Sparse activation** - ReLU outputs a zero value for negative inputs, which leads to sparse activations. This means that only a subset of neurons are activated at any given time, making the network more efficient.
- ❖ ReLU helps to mitigate the vanishing gradient problem, allowing neural networks to learn more effectively from backpropagation.
- ❖ **Convergence:** ReLU accelerates the convergence of stochastic gradient descent compared to the sigmoid and tanh functions.
- ❖ **Training:** ReLU trains quickly and efficiently

Disadvantages of the ReLU Activation Function

- ❖ The main issue with ReLU is that all the negative values become zero immediately, which decreases the ability of the model to fit or train from the data properly.

- ❖ ReLU also has some problems, such as **exploding gradient**. Exploding gradient occurs when large error gradients accumulate and result in very large updates to neural network model weights during training.
- ❖ ReLU is common because it's simple to implement and effective at overcoming the limitations of other previously popular activation functions, such as Sigmoid
- ❖ The main advantage of using the ReLU function over other activation functions is that **it does not activate all the neurons at the same time.**
- ❖ **This means that the neurons will only be deactivated if the output of the linear transformation is less than 0.**
- ❖ The plot below will help you understand this better-
- ❖ $f(x)=\max(0,x)$



- ❖ For the negative input values, the result is zero, which means the neuron does not get activated. Since only a certain number of neurons are activated, the ReLU function is far more computationally efficient when compared to the **sigmoid and tanh function**. Here is the python function for ReLU:

```
def relu_function(x):
```

```
    if x<0:
```

```
        return 0
```

```
    else:
```

```
        return x
```

```
relu_function(7), relu_function(-7)
```

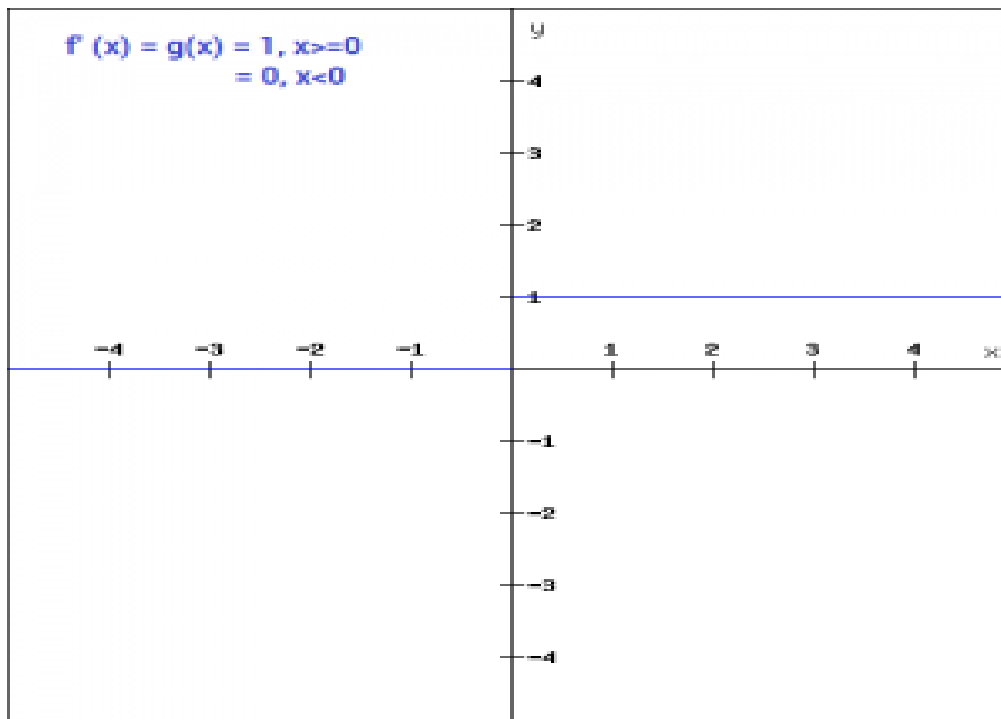
Output:

(7, 0)

❖ Let's look at the gradient of the ReLU function.

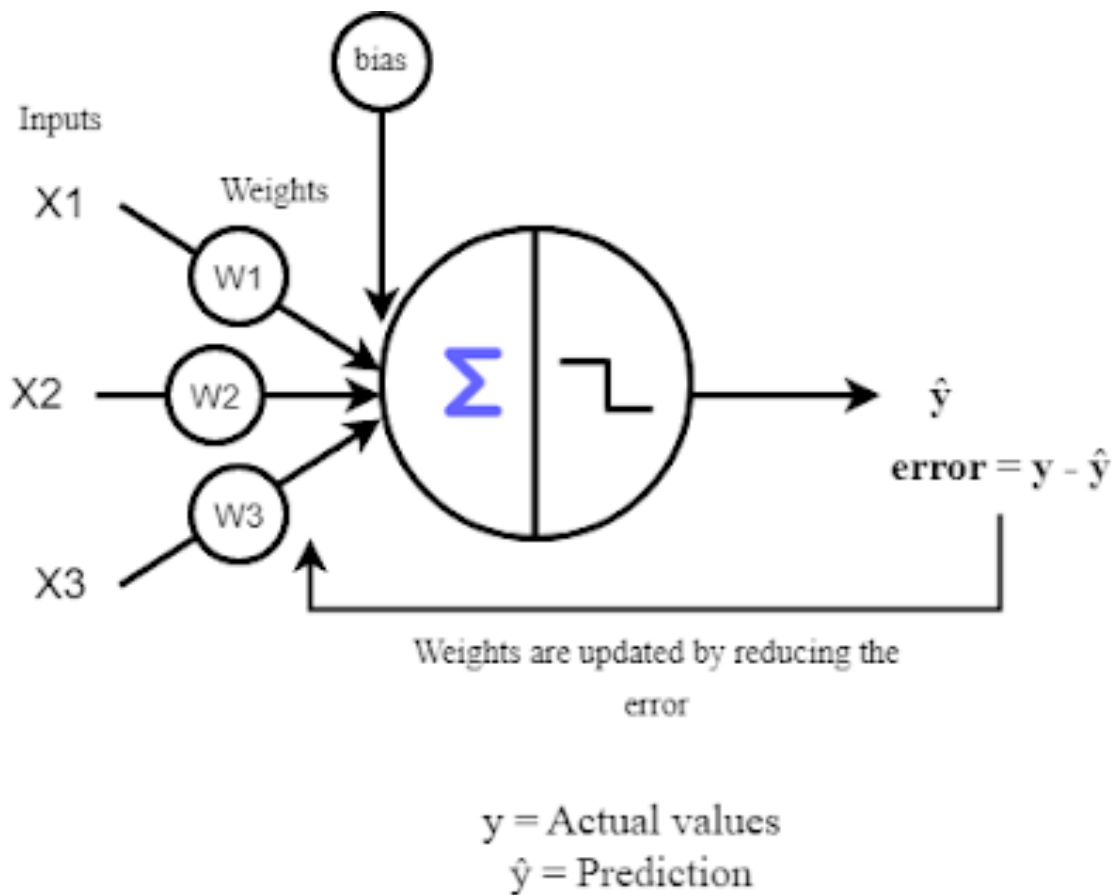
$$f'(x) = 1, x \geq 0$$

$$= 0, x < 0$$



4. What is a Perceptron & explain its components with a diagram?

- ❖ A perceptron is one of the simplest artificial neural network architectures, and it serves as the fundamental building block of more complex neural networks.
- ❖ It was developed by Frank Rosenblatt in the late 1950s.
- ❖ **A perceptron is primarily used for binary classification tasks, where it takes a set of input values, applies weights to those inputs, sums them up, and then passes the result through an activation function to produce an output. This output is typically either 0 or 1, making it suitable for binary decisions.**
- ❖ The whole point of training the perceptron is to adjust the weights of each input according to the training data.
- ❖ **For training the perceptron to find the best weights, Frank Rosenblatt proposed an algorithm that is largely inspired by Hebb's rule.**
- ❖ **Hebb's rule states that when a biological neuron triggers another neuron, the connection between these two neurons grows stronger.**
- ❖ **There is an awesome phrase for this property of neurons, ie, Neurons that fire together wire together.**
- ❖ Inspired by Hebb's rule, the Perceptron can be trained by finding the error made by the algorithm and finding the weights that reduce this error as much as possible.
- ❖ This is the main goal of training a machine learning algorithm in general.
- ❖ The algorithm will first make a prediction based on the training data given, then this prediction is compared with the actual desired output.
- ❖ The error is calculated for each of the training instances by comparing the prediction and actual output, finally, this error is reduced during the training process and the weights corresponding to the correct predictions are adjusted accordingly.



❖ The learning(training) rule can be written as

$$w_{i,j}(\text{next}) = w_{i,j} + \eta(y_i - \hat{y})x_i$$

- ❖ Where $w_{i,j}$ is the weights between i th input neuron and j th output neuron
- ❖ η (eta) is the learning rate
- ❖ y is the target output or the actual values
- ❖ $\hat{y}$ is the predicted value of the output neuron
- ❖ x_i is the input value of the current training instance
- ❖ The algorithm is being trained by updating the weights until getting the optimal results.
- ❖ The learning rate(η) is required to determine the length of each step taken for finding optimal weights. Sometimes, it is better to have a smaller learning rate to get a minimum error, but sometimes a large learning rate is also preferred.

Basic Components of Perceptron

- ❖ Mr. Frank Rosenblatt invented the perceptron model as a binary classifier which contains three main components. The basic components of a perceptron are:
 - ❖ **Input nodes:** Also known as the input layer, these nodes accept the initial data for processing. Each input node has a real numerical value.
 - ❖ **Weights:** Represent the strength of the connection between units.
 - ❖ **Bias:** A part of the perceptron.
 - ❖ **Weighted sum:** A function within the neuron that produces an output based on the numerical value of the inputs and the weights
 - ❖ **Activation function:** Applies to the input to produce a binary output. Activation functions are important for building multilayer perceptrons and can add non-linearity to the structure.
- These are the final and important components that help to determine whether the neuron will fire or not.

